Commercial Embedding APIs — A Practitioner's Guide

How the major LLM providers offer embeddings, how they compare, and what this means for LLM agent memory systems. Current as of May 2026.

TL;DR (one screen)

The landscape. Five serious commercial embedding providers in 2026: **OpenAI**, **Google**, **Voyage AI**, **Cohere**, and **Jina**. Anthropic does *not* offer embeddings — it explicitly recommends Voyage AI as its partner. Mistral has an embedding model but plays a minor role.

Pricing has collapsed. From $0.10–0.20 per million tokens being normal in 2023, the floor is now **Google text-embedding-005 at $0.006/M** — a 20× drop. OpenAI sits at $0.02–$0.13, Voyage charges a premium of $0.18 for its best model.

The big technical shift since 2023. Three things changed at once:

1. **Matryoshka embeddings** — one model produces an embedding that can be safely truncated to lower dimensions without retraining. Lets you trade quality for storage at inference time.
2. **Native multimodal** — Cohere Embed v4 and Voyage Multimodal embed text and images into a shared vector space.
3. **Specialised models** — code, multilingual, legal/medical are now first-class product tiers rather than one-size-fits-all.

For LLM agent memory. The right choice depends on whether you need cheap and good (Google), best-in-class English (Voyage 3-large), ecosystem fit (OpenAI), multilingual (Cohere/Jina), or local control (open-weight models like BGE, E5). For most memory systems shipping in 2026, **OpenAI text-embedding-3-large with** `dimensions=1024` or **Google text-embedding-005** are the safe defaults.

---

1. What an embedding actually is

An embedding is a fixed-length vector of floating-point numbers (typically 256–4096 dimensions) that represents the "meaning" of a piece of text in such a way that semantically similar texts produce similar vectors. Similarity is usually measured by cosine distance.

For LLM agent memory, embeddings are the standard way to **store** memory items (each turn or fact becomes a vector) and **retrieve** them at query time (the question is embedded, top-K nearest stored vectors are returned). This is the backbone of every vector-DB-based memory system: HippoRAG, Mem0, A-Mem, and effectively every "ChatGPT memory" implementation.

The quality of the embedding model directly bounds the quality of the memory system. If the embedding cannot distinguish two semantically different memories, retrieval will mix them. If it cannot recognise that two paraphrased statements mean the same thing, retrieval will miss.

---

2. How a commercial embedding API works

The interface is uniformly simple across providers. Send text, get a vector. A typical call:

```
POST /embeddings
{
  "model": "text-embedding-3-large",
  "input": ["string 1", "string 2", ...],
  "dimensions": 1024          # optional, providers vary
}

Response:
{
  "data": [
    { "embedding": [0.012, -0.034, ...], "index": 0 },
    { "embedding": [...], "index": 1 }
  ],
  "usage": { "prompt_tokens": 42 }
}
```

Three things to know:

- **Pricing is per input token only** — embeddings have no output token concept.
- **Batching matters** — sending 100 strings in one call is far cheaper and faster than 100 calls.
- **Rate limits** are usually generous compared to chat APIs, but not infinite.

---

3. The five major providers

3.1 OpenAI

Model	Dimensions	Max tokens	Price ($/M)	MTEB
text-embedding-3-small	1536 (truncatable)	8191	$0.02	62.3
text-embedding-3-large	3072 (truncatable)	8191	$0.13	64.6

Distinctive features:

- **Matryoshka support via `dimensions` parameter.** Set `dimensions=1024` and the model returns a truncated, normalised vector that still performs nearly as well as the full 3072. Massive storage win.
- **Ecosystem leadership.** Every vector DB, every framework, every notebook example uses these by default. The path of least resistance.
- **Batch API at 50% discount** ($0.01/M small, $0.065/M large) for non-interactive jobs.

Weaknesses:

- 8K-token input is short compared to Voyage's 32K — long documents must be chunked.
- No native multimodal text+image.

When to use: general-purpose RAG/memory, anything where ecosystem integration matters, anything that benefits from variable dimensions at storage time.

3.2 Google (Vertex AI / Gemini API)

Model	Dimensions	Max tokens	Price ($/M)
text-embedding-005	768 (Matryoshka)	2048	$0.006
text-multilingual-embedding-002	768	2048	$0.006
gemini-embedding (newest)	up to 3072	longer	tier-dependent

Distinctive features:

- **Cheapest serious option.** $0.006/M is roughly 20× under most competitors. For a memory system embedding millions of conversation turns per month, this is a step-change in cost.
- **Task-type parameter.** You specify whether you are embedding for retrieval-document, retrieval-query, classification, similarity, etc. The model produces optimised vectors per task type — a subtle but real quality lift.
- **Strong multilingual coverage** with the multilingual-002 variant.

Weaknesses:

- Smaller default dimensions (768) cap absolute quality vs OpenAI/Voyage.
- Vertex AI auth complexity vs OpenAI's plain API key.
- Documentation churn — model names and behaviours have shifted multiple times since 2024.

When to use: cost-sensitive deployments, multilingual content, anything already on Google Cloud.

3.3 Voyage AI (Anthropic's official embedding partner)

Model	Dimensions	Max tokens	Price ($/M)	Specialty
voyage-3-large	1024 (Matryoshka)	32000	$0.18	Best general English
voyage-3	1024	32000	$0.06	Balanced
voyage-code-3	1024	32000	$0.18	Code
voyage-law-2	1024	16000	$0.12	Legal
voyage-multimodal-3	1024	varies	tier	Text + images

Distinctive features:

- **32K input tokens** — 4× OpenAI's 8K. Whole documents fit directly; less chunking, less retrieval drift.
- **Top of MTEB leaderboard.** Voyage-3-large leads at ~65.4% MTEB, narrowly ahead of OpenAI.
- **Domain-specialised models.** voyage-code-3 outperforms general-purpose embeddings by 4+ MTEB points on code retrieval.
- **Anthropic's official recommendation** — the Anthropic docs link directly to Voyage. If you're building a Claude-backed agent and care about ecosystem consistency, this is the path.

Weaknesses:

- Most expensive of the major providers ($0.18 vs Google's $0.006 — 30× more).
- Smaller ecosystem; fewer pre-built integrations.

When to use: when quality matters more than cost; long-document retrieval; code RAG; Claude-centric stacks.

3.4 Cohere

Model	Dimensions	Max tokens	Price ($/M)	Specialty
embed-english-v3.0	1024 (Matryoshka)	512	$0.10	English
embed-multilingual-v3.0	1024	512	$0.10	100+ languages
embed-v4	configurable	varies	$0.12 text / $0.47 image	Multimodal, multilingual

Distinctive features:

- **Native multimodal in embed-v4** — text and images into one shared space at the model level.
- **Best-in-class for enterprise multilingual.** 100+ languages with consistent quality, not just English-plus-translation.
- **Rerankers as a separate product.** Cohere Rerank is widely used as a second-stage scorer on top of embeddings from any provider — many production RAG stacks pair OpenAI embeddings + Cohere reranker.

Weaknesses:

- 512-token max input on v3 is very short.
- Image pricing on v4 is steep ($0.47/M).

When to use: multilingual, enterprise compliance settings, any pipeline that benefits from a separate reranker.

3.5 Jina AI

Model	Dimensions	Max tokens	Price ($/M)	Specialty
jina-embeddings-v3	1024 (Matryoshka)	8192	$0.02	Multilingual budget
jina-clip-v2	1024	varies	tier	Text + image

Distinctive features:

- **Aggressively priced multilingual.** Jina-v3 is the cheap multilingual choice.
- **Open-weight versions** also published — same architecture is available for self-hosting.
- **Task-LoRA adapters.** A single base model with small task-specific adapters for retrieval, classification, separation. Lets one model serve multiple use-cases.

When to use: budget multilingual; teams that want a clean self-hosted-or-API choice.

3.6 Mistral, Nomic, others

- **Mistral Embed** — small ecosystem footprint; reasonable quality, decent price ($0.10/M), 8K context.
- **Nomic Embed** — open-weight, MIT licence, full reproducible training data; popular for teams who refuse to depend on closed APIs.

Open-weight models (BGE, E5, Nomic, Stella) deserve their own section in the appendix — for many production memory systems, self-hosting an open-weight embedding model is the right call once volume crosses a threshold.

---

4. Cross-provider comparison

4.1 Capability matrix

Capability	OpenAI	Google	Voyage	Cohere	Jina
Matryoshka (variable dims at inference)	Yes	Yes	Yes	Yes	Yes
Native multimodal (text+image)	–	Partial	voyage-multimodal-3	embed-v4	jina-clip-v2
Long input (≥16K tokens)	–	–	32K	–	8K
Strong multilingual	–	Yes	–	Yes	Yes
Code-specialised	–	–	voyage-code-3	–	–
Task-type conditioning	–	Yes	–	–	Yes (LoRA)

Capability	OpenAI	Google	Voyage	Cohere	Jina
Reranker as separate product	–	–	Yes	Yes	Yes
Open-weight version	–	–	–	–	Yes
Batch API discount	50% off	–	–	–	–

4.2 Pricing per million tokens (May 2026, approximate)

Provider / Model	Price	Relative
Google text-embedding-005	$0.006	1×
OpenAI text-embedding-3-small (batch)	$0.01	~1.7×
OpenAI text-embedding-3-small	$0.02	~3.3×
Jina embeddings v3	$0.02	~3.3×
Voyage-3	$0.06	~10×
Cohere embed-v3	$0.10	~17×
Mistral Embed	$0.10	~17×
Cohere embed-v4 (text)	$0.12	~20×
OpenAI text-embedding-3-large	$0.13	~22×
Voyage-3-large	$0.18	~30×

4.3 What ~$1 of embedding buys

At 100 tokens per memory item (a short fact or chat turn):

- Google: ~16.7M memory items embedded per dollar.
- OpenAI small: 5M items.
- OpenAI large: 770K items.
- Voyage-3-large: 555K items.

For a system embedding 10M items per month, this is the difference between $6 and $1,800. The choice matters more than people often acknowledge.

---

5. Concepts that matter for LLM agent memory

5.1 Matryoshka embeddings — the most important 2024 innovation

Matryoshka Representation Learning is a training technique where the model is trained so that the first k dimensions of the full vector themselves form a usable embedding. You can store a 3072-dim vector once and at query time decide to use only the first 256, 512, or 1024 — without retraining anything. This decouples *storage cost* from *retrieval quality*.

In practice, for memory systems:

- Index at full dimension (best recall).
- Query at lower dimension if cost or latency matters.
- For tiered memory: store hot memory at full, cold memory truncated.

Both OpenAI and Voyage support this via the `dimensions` API parameter. Google supports it on newer models. This single feature changes the cost-quality trade-off curve fundamentally.

5.2 Task-type conditioning

Google's API takes a `task_type` parameter (RETRIEVAL_QUERY, RETRIEVAL_DOCUMENT, SEMANTIC_SIMILARITY, CLASSIFICATION, ...). The model produces a slightly different embedding optimised for that role. For agent memory, this means:

- Embed memory entries with `RETRIEVAL_DOCUMENT`.
- Embed user queries with `RETRIEVAL_QUERY`.
- The asymmetric query/document embedding pair often beats symmetric same-model embedding for retrieval.

OpenAI does not expose this explicitly but their model is trained to handle both. Voyage offers it on some endpoints.

5.3 Rerankers — the second-stage trick

Embedding-based retrieval gives top-K candidates fast but imprecisely. A *reranker* takes those K candidates and scores them more carefully using a cross-encoder (which actually attends jointly over query + candidate). Cohere Rerank, Voyage Reranker, and Jina Reranker are the main commercial options.

Standard production pattern:

1. Embed the query, retrieve top-50 from vector DB.
2. Pass query + 50 candidates to a reranker.
3. Keep top-5 by rerank score.

This is the single biggest practical quality lift for RAG/memory systems, and it is often missing in research-paper architectures.

5.4 The asymmetry of query and document length

A memory item might be a single fact ("user prefers terse responses"). A query might be a full multi-sentence question. The mismatch in length hurts cosine similarity. Voyage's 32K input is partly an attempt to handle this; Cohere's separate query/document models handle it differently. For any memory system, validate your retrieval quality on the *actual* shape of your queries and memories — benchmarks

rarely match.

5.5 Embedding drift

If you change embedding models, every previously stored vector becomes incompatible. Your entire memory store must be re-embedded — a massive cost on real systems. This is one of the central problems for long-lived agent memory:

- A-Mem (covered earlier in the notes) is bottlenecked by static embeddings.
- Chain-of-Memory does *not* solve this despite its name.
- No commercial provider currently offers a stable-across-versions guarantee.

In practice: pick a provider you can commit to, version your memory store with the embedding model identifier, and budget for re-embedding when you upgrade.

---

6. Recommendations by use-case

Use-case	Recommendation
Default modern memory system	OpenAI text-embedding-3-large at `dimensions=1024`
Cost-sensitive at scale	Google text-embedding-005
Claude-backed agent	Voyage-3-large (Anthropic's official recommendation)
Code memory or code RAG	Voyage-code-3
Legal/medical memory	Voyage-law-2 or fine-tuned open-weight
Multilingual production	Cohere embed-v4 or Jina v3
Multimodal (text + images) memory	Cohere embed-v4 or Voyage multimodal-3
Long-document retention	Voyage (32K input)
Self-hosted, no cloud dependency	Open-weight BGE-M3, E5-Mistral, Nomic-v1.5, Jina-v3
Production RAG with reranker	OpenAI/Voyage embeddings + Cohere Rerank

---

7. Implications for LLM agent memory specifically

7.1 The static-embedding bottleneck (cross-link to notes.md A-Mem section)

Every commercial embedding model is *frozen at training time*. Memories embedded today and queried in two years will be searched with the same vector. If user vocabulary drifts, domain terminology evolves, or the world changes, the embedding cannot adapt. Three partial mitigations:

1. **Periodic re-embedding** of important memories with the latest model.
2. **Hybrid retrieval** — combine semantic embedding match with keyword/BM25 match so vocabulary changes do not break recall.
3. **LLM-generated structured attributes** alongside dense vectors (the A-Mem approach) — tags and links survive embedding model changes.

7.2 Choosing dimensions for memory

- 256 dimensions: viable only with Matryoshka models; saves 12× storage vs 3072. Use for very large memory stores where storage cost dominates.
- 768–1024 dimensions: standard for serious memory systems. Best quality-to-cost ratio.
- 3072 dimensions: rarely worth the storage cost in practice unless you have very fine semantic distinctions to make.

7.3 The reranker-as-importance-scorer pattern

A reranker scores (query, candidate) pairs. The same machinery can be used to score (current context, candidate memory) pairs for *retrieval ranking* in a memory system — which is exactly what Generative Agents' importance-weighting prompt does, but more cheaply. Underexplored in the agent memory literature.

7.4 Provider choice as a strategic lock-in

Embedding models are not interchangeable across providers — switching providers requires re-embedding the entire memory store. This is a strategic lock-in that few teams budget for. The right default is to:

1. Pick a provider you are comfortable depending on for years.
2. Version your memory entries with the embedding model name.
3. Build a re-embedding pipeline before you need it.

---

8. Open challenges with current commercial embedding APIs

1. **No stable-across-versions guarantee.** When a provider updates a model, your stored embeddings silently degrade.
2. **No native incremental fine-tuning.** You cannot adapt the embedding model to your domain via API; you can only use the pre-trained vectors. (Cohere allows custom fine-tuning, but it is opaque and expensive.)
3. **No long-term-memory-optimised embedding.** All commercial models are trained for general retrieval; none are explicitly trained to encode multi-month dialogue history or evolving user preferences. This is an underserved niche.
4. **Limited transparency.** Training data, evaluation harnesses, and dimension allocation strategies are mostly undisclosed for closed models. You cannot reason about why retrieval failed on a specific case.

5. **Multimodal is still early.** Cohere v4 and Voyage Multimodal are the first credible attempts, but image-text alignment quality lags pure text models by a noticeable margin.
6. **Embedding-as-a-service economics.** At 100M+ tokens per month, self-hosting an open-weight model is often cheaper than any API. The break-even point shifts the architecture decision.
7. **Embedding evaluation is task-specific.** MTEB benchmarks reflect generic retrieval. For agent memory specifically — multi-session, paraphrased, user-personalised — there is no good benchmark and providers do not publish targeted scores.

---

9. Quick-start: API code samples

OpenAI

```
from openai import OpenAI
client = OpenAI()
resp = client.embeddings.create(
    model="text-embedding-3-large",
    input=["memory item text"],
    dimensions=1024,
)
vec = resp.data[0].embedding
```

Voyage AI

```
import voyageai
vo = voyageai.Client()
result = vo.embed(
    ["memory item text"],
    model="voyage-3-large",
    input_type="document",     # or "query"
)
vec = result.embeddings[0]
```

Cohere

```
import cohere
co = cohere.ClientV2()
resp = co.embed(
    model="embed-v4.0",
    texts=["memory item text"],
    input_type="search_document",   # or "search_query"
)
vec = resp.embeddings.float[0]
```

Google

```
from google import genai
client = genai.Client()
result = client.models.embed_content(
    model="text-embedding-005",
    contents="memory item text",
    config={"task_type": "RETRIEVAL_DOCUMENT"},
)
vec = result.embeddings[0].values
```

---

10. Bibliography

- **OpenAI embeddings overview** — https://platform.openai.com/docs/guides/embeddings
- **OpenAI text-embedding-3 release** — https://openai.com/index/new-embedding-models-and-api-updates/
- **Anthropic embeddings doc** — https://docs.claude.com/en/docs/build-with-claude/embeddings (recommends Voyage)
- **Voyage AI docs** — https://docs.voyageai.com
- **Cohere embed docs** — https://docs.cohere.com/docs/embeddings
- **Google Vertex AI embeddings** — https://cloud.google.com/vertex-ai/generative-ai/docs/embeddings
- **Jina AI embeddings** — https://jina.ai/embeddings
- **MTEB benchmark leaderboard** — https://huggingface.co/spaces/mteb/leaderboard
- **Matryoshka Representation Learning paper** — https://arxiv.org/abs/2205.13147